

EDS CodeTantra Lab Course

Name: Akshara Rodge

Batch: CH-2

PRN: 202501050040

Practical-3

3.1 Lab Assignments

3.1.1 Numpy array operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `rows` and `cols`, representing the number of rows and the number of columns.
- The next `rows` lines contain `cols` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Program:

```
import numpy as np
```

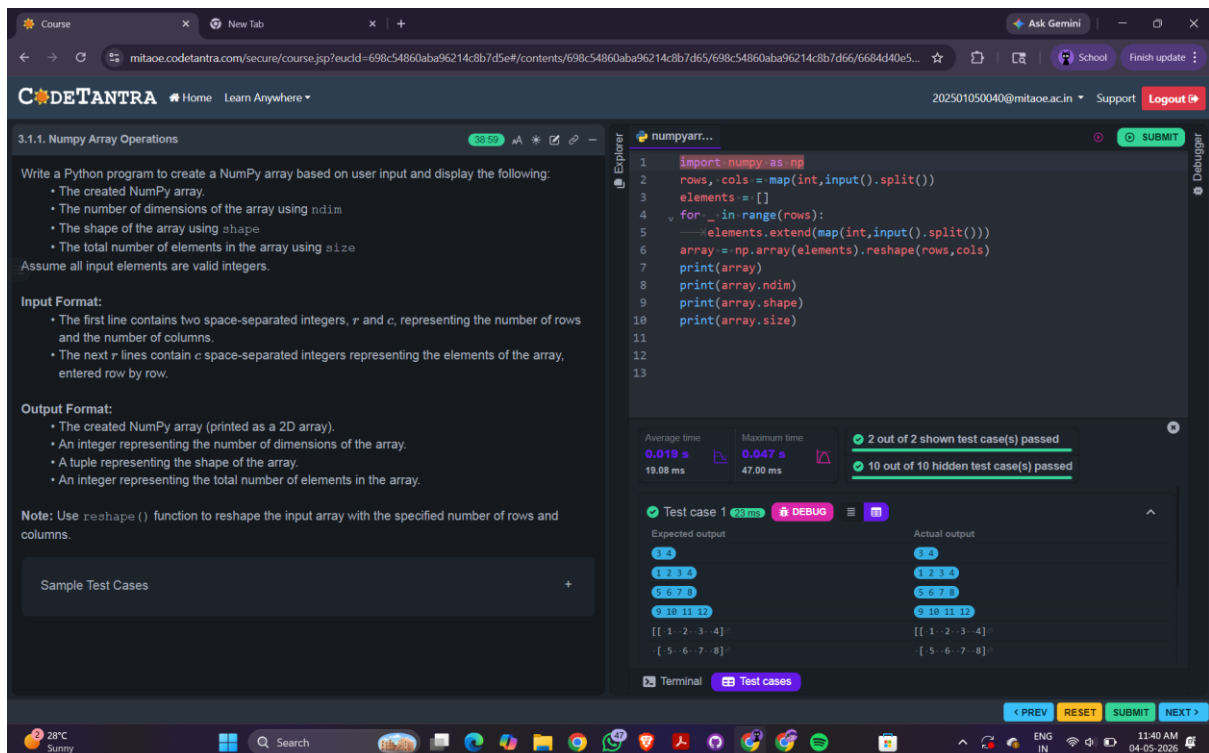
```
rows, cols = map(int,input().split())
```

```

elements = []
for _ in range(rows):
    elements.extend(map(int,input().split()))
array = np.array(elements).reshape(rows,cols)
print(array)
print(array.ndim)
print(array.shape)
print(array.size)

```

Screenshot:



3.2 Lab Assignments

3.2.1 Numpy: Matrix operations

The given code takes two matrices, A and B , as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition ()**
2. **Subtraction ()**
3. **Element-wise Multiplication ()**
4. **Matrix Multiplication ()**
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for , each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for , each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Program:

```
import numpy as np

# Input matrices

print("Enter Matrix A:")

matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")

matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition

addition_result=matrix_a+matrix_b

print("Addition (A + B):")

print(addition_result)

# Subtraction

subtraction_result=matrix_a-matrix_b
```

```
print("Subtraction (A - B):")
print(subtraction_result)
# Multiplication (element-wise)
elementwise_multiplication_result=matrix_a*matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)
# Matrix multiplication (dot product)
matrix_multiplication_result=np.dot(matrix_a,matrix_b)
print("A dot B:")
print(matrix_multiplication_result)
# Transpose
transpose_a=matrix_a.T
print("Transpose of A:")
print(transpose_a)
```

Screenshot:

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:
You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a · matrix_b`)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:
The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

```

1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 addition_result = matrix_a + matrix_b
13 print("Addition (A + B):")
14 print(addition_result)
15
16 # Subtraction
  
```

Test case 1: Expected output: Enter Matrix A: 1 2 3, 4 5 6, 7 8 9; Enter Matrix B: 1 1 1. Actual output: Enter Matrix A: 1 2 3, 4 5 6, 7 8 9; Enter Matrix B: 1 1 1.

3.2.2 Numpy: horizontal and vertical stacking

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3×3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Program:

```
import numpy as np
```

```
# Input matrices
```

```

print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])
# Perform horizontal stacking (hstack)
h_stack = np.hstack((arr1, arr2))
# Perform vertical stacking (vstack)
v_stack = np.vstack((arr1, arr2))
# Output results
print("Horizontal Stack:")
print(h_stack)
print("Vertical Stack:")
print(v_stack)

```

Screenshot:

The screenshot shows a web-based IDE interface for a course titled "3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays". The code editor displays a Python script that takes two 3x3 arrays as input, performs horizontal and vertical stacking, and prints the results. The terminal shows the execution output, and the test results panel indicates that all test cases passed.

Code:

```

1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 h_stack = np.hstack((arr1, arr2))
12
13 # Perform vertical stacking (vstack)
14 v_stack = np.vstack((arr1, arr2))
15

```

Test Results:

- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1:

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9

3.2.3 Numpy: custom sequence generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Program:

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
sequence = np.arange(start, stop, step)

# Print the generated sequence
print(sequence)
```

Screenshot:

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

```

1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 sequence = np.arange(start, stop, step)
10
11 # Print the generated sequence
12 print(sequence)
  
```

Average time: 0.023 s, Maximum time: 0.056 s, 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1: Expected output [3 5 7 9], Actual output [3 5 7 9].

Test case 2: Expected output [3 5 7 9], Actual output [3 5 7 9].

3.2.4 Numpy: arithmetic

You are given two arrays A and B . Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A .

3. Bitwise Operations:

- Perform bitwise `AND`, bitwise `OR`, and bitwise `XOR` on the arrays (ex: $A_1 \text{ OR } B_1$).

Input Format:

- The first line contains space-separated integers representing the elements of array A .
- The second line contains space-separated integers representing the elements of array B .

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Program:

```
import numpy as np

def array_operations(A, B):

    # Convert A and B to NumPy arrays

        A = np.array(A)

        B = np.array(B)

    # Arithmetic Operations

        sum_result = A + B

        diff_result = A - B

        prod_result = A * B

    # Statistical Operations

        mean_A = np.mean(A)

        median_A = np.median(A)

        std_dev_A = np.std(A)

    # Bitwise Operations

        and_result = np.bitwise_and(A, B)

        or_result = np.bitwise_or(A, B)

        xor_result = np.bitwise_xor(A, B)

    # Output results with one space between each element

        print("Element-wise Sum:", ' '.join(map(str, sum_result)))

        print("Element-wise Difference:", ' '.join(map(str, diff_result)))

        print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```

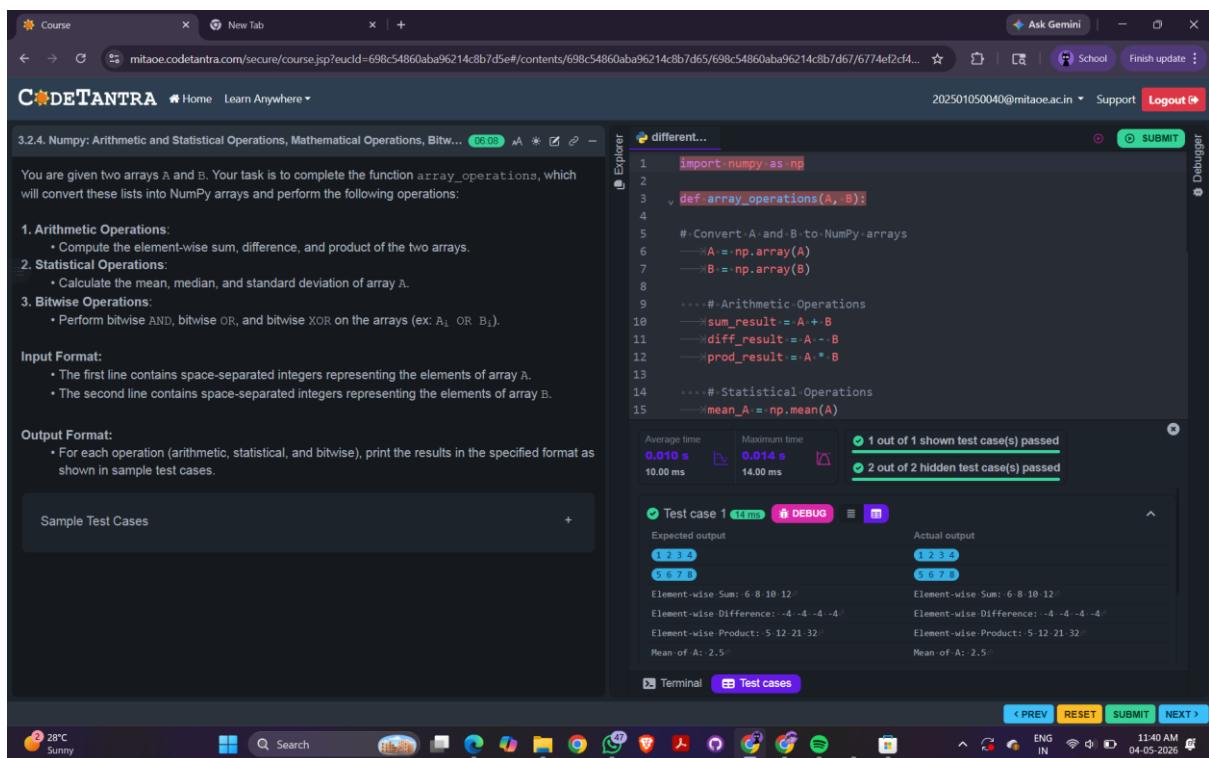
print(f"Mean of A: {mean_A}")
print(f"Median of A: {median_A}")
print(f"Standard Deviation of A: {std_dev_A}")
print("Bitwise AND:", ' '.join(map(str, and_result)))
print("Bitwise OR:", ' '.join(map(str, or_result)))
print("Bitwise XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B

array_operations(A, B)

```

Screenshot:



3.2.5 Numpy: Copying and viewing arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Program:

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99

print("Original array after modifying view:", original_array)
print("View array:", view_array)

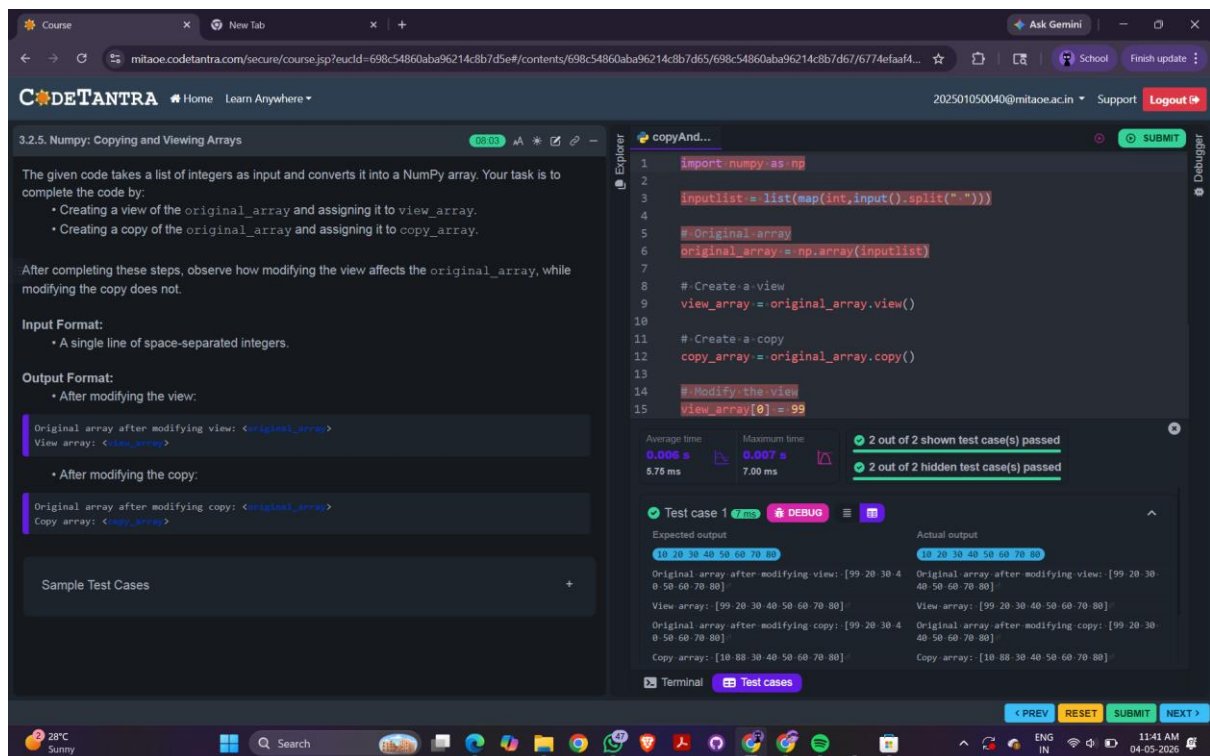
# Modify the copy
```

```
copy_array[1] = 88
```

```
print("Original array after modifying copy:", original_array)
```

```
print("Copy array:", copy_array)
```

Screenshot:



3.2.6 Numpy: searching

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Program:

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
search_indices = np.where(array1 == search_value)
print(search_indices[0])

# Count occurrences in array1
count_occurrences = np.count_nonzero(array1 == count_value)
print(count_occurrences)

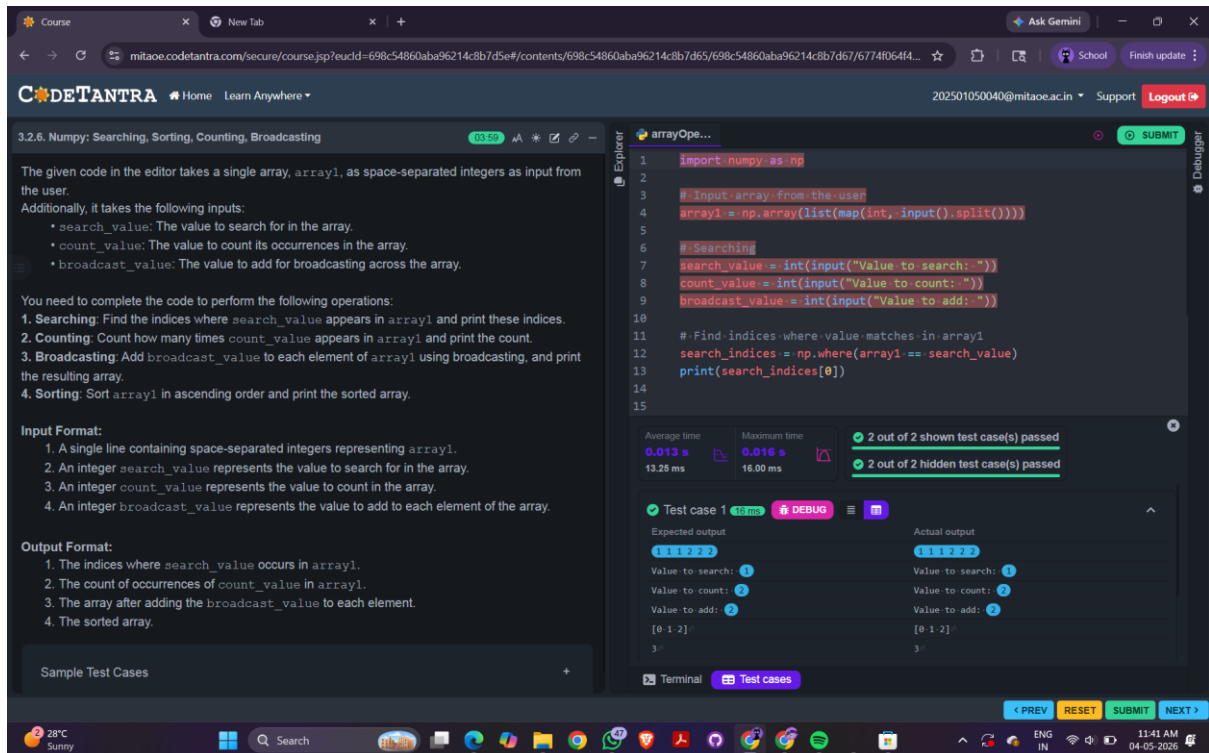
# Broadcasting addition
broadcasted_array = array1 + broadcast_value
print(broadcasted_array)
```

Sort the first array

```
sorted_array = np.sort(array1)
```

```
print(sorted_array)
```

Screenshot:



3.2.7 Student Data analysis

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.

- **Find average marks of Subject 1 and Subject 2**: Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3**: Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3**: Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2**: Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2**: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1**: Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2**: Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject**: Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90** : Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order**: Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1**: Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1**: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Program:

```
import numpy as np
```

```
a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
```

```
# 1. Print all student details
```

```
print("All student Details:\n", a)
```

```
# 2. Find total students
```

```
total_students = a.shape[0]
```

```
print("Total Students:", total_students)
```

3. Print all student roll numbers

```
roll_numbers = a[:, 0]
```

```
print("All Student Roll Nos", roll_numbers)
```

4. Print Subject 1 marks

```
subject_1_marks = a[:, 1]
```

```
print("Subject 1 Marks", subject_1_marks)
```

5. Find minimum marks in Subject 2

```
min_marks_subject_2 = np.min(a[:, 2])
```

```
print("Min marks in Subject 2", round(min_marks_subject_2, 1))
```

6. Find maximum marks in Subject 3

```
max_marks_subject_3 = np.max(a[:, 3])
```

```
print("Max marks in Subject 3", round(max_marks_subject_3, 1))
```

7. Print all subject marks

```
print("All subject marks:", a[:, 1:])
```

8. Find total marks of students

```
total_marks = np.sum(a[:, 1:], axis=1)
```

```
print("Total Marks", total_marks)
```

9. Find average marks of each student

```
average_marks_per_student = np.mean(a[:, 1:], axis=1)
```

```
print(np.round(average_marks_per_student,1))
```

10. Find average marks of each subject

```
average_marks_per_subject = np.mean(a[:, 1:], axis=0)
```

```
print(f"Average Marks of each subject {[round(i, 1) for i in  
average_marks_per_subject]}".replace(",",""))
```

11. Find average marks of Subject 1 and Subject 2

```
average = np.average(a,axis=0)
```

```
print("Average Marks of S1 and S2",average[1:3])
```

12. Find average marks of Subject 1 and Subject 3

```
average_s1_s3 = np.mean(a[:, [1, 3]], axis=1)
```

```
print("Average Marks of S1 and S3", average[1::2])
```

13. Find roll number of the student with maximum marks in Subject 3

```
roll_max_s3 = roll_numbers[np.argmax(a[:, 3])]
```

```
print("Roll no who got maximum marks in Subject 3", roll_max_s3)
```

14. Find roll number of the student with minimum marks in Subject 2

```
roll_min_s2 = roll_numbers[np.argmin(a[:, 2])]
print("Roll no who got minimum marks in Subject 2", roll_min_s2)
```

15. Find roll numbers of students who scored 24 marks in Subject 2

```
roll_24_marks_s2 = roll_numbers[a[:, 2] == 24]
print("Roll no who got 24 marks in Subject 2",
      np.array([roll_24_marks_s2.tolist()]))
```

16. Find count of students who got less than 40 marks in Subject 1

```
count_s1_less_40 = np.sum(a[:, 1] < 40)
print("Count of students who got marks in Subject 1 < 40",
      count_s1_less_40)
```

17. Find count of students who got more than 90 marks in Subject 2

```
count_s2_greater_90 = np.sum(a[:, 2] > 90)
print("Count of students who got marks in Subject 2 > 90:",
      count_s2_greater_90)
```

18. Find count of students who scored ≥ 90 in each subject

```
count_subjects_greater_90 = np.sum(a[:, 1:] >= 90, axis=0)
print("Count of students in each subject who got marks  $\geq 90$ :",
      count_subjects_greater_90)
```

19. Find count of subjects in which each student scored ≥ 90

```
print("Roll no:", a[:, 0])
count_subjects_per_student = np.sum(a[:, 1:] >= 90, axis=1)
```

```
print("Count of subjects in which student got marks >= 90:",  
count_subjects_per_student)
```

20. Print Subject 1 marks in ascending order

```
sorted_s1_marks = np.sort(subject_1_marks)
```

```
print(sorted_s1_marks)
```

21. Print students who scored between 50 and 90 in Subject 1

```
students_s1_50_to_90 = a[(a[:,1] >= 50) & (a[:,1] <= 90)]
```

```
print(students_s1_50_to_90)
```

22. Find index positions of students who scored 79 in Subject 1

```
index_positions_s1_79 = np.where(subject_1_marks == 79)[0]
```

```
print(a)
```

```
print((index_positions_s1_79,))
```

Screenshot:

The screenshot displays a web-based IDE environment. On the left, a sidebar lists various tasks for '3.2.7. Student Data Analysis and Operations'. The main area shows a Python script that reads a CSV file named 'Sample.csv' and performs several operations: printing all student details, finding the total number of students, and printing marks for three subjects. The script is executed, and the output is shown in the terminal. A test cases panel on the right indicates that 1 out of 1 shown test case(s) passed.

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5
6 # 1. Print all student details
7 print("All student Details:\n", a)
8
9 # 2. Find total students
10 total_students = a.shape[0]
11 print("Total Students:", total_students)
12
13 # 3. Print all student roll numbers
14 roll_numbers = a[:, 0]
15 print("All Student Roll Nos", roll_numbers)
```

Test case 1 passed

Expected output	Actual output
All student Details: [[301, 67, 77, 88]] [[302, 78, 88, 77]] [[303, 45, 56, 89]] [[304, 88, 98, 45]] [[305, 78, 88, 99]]	All student Details: [[301, 67, 77, 88]] [[302, 78, 88, 77]] [[303, 45, 56, 89]] [[304, 88, 98, 45]] [[305, 78, 88, 99]]

